

PRACTICAL 12

CI/CD Pipeline with GitHub Actions

Student Name: Prisha Kalola

Student ID: 24DIT026

Semester: 5th Semester

CO/PO: CO5 / PO3, PO5

Course: Advanced Web Development Framework (ITUE301)

1. Aim

To automate testing and build verification of the application using a CI/CD pipeline built with GitHub Actions.

2. Objectives

- Set up a GitHub Actions workflow (.github/workflows/ci.yml) triggering on git push to main.
- Automate dependency installation (npm install) and Jest unit testing (npm test).
- Demonstrate a failed CI pipeline caused by a broken test.
- Demonstrate a passed CI pipeline after fixing the test.

3. Connection with Previous Practicals

Practical 12 automates testing and build validation for code written in Practicals 4-11.

4. Prerequisites

- Practicals 4-11 completed repository pushed to GitHub.
- Installed dev-dependencies: jest, supertest.

5. Theory Concepts

5.1 Continuous Integration (CI)

Continuous Integration automatically builds and tests code changes on every commit push. GitHub Actions executes workflow YAML jobs on cloud virtual environments (e.g. ubuntu-latest).

6. Problem Definition

Create Jest test in tests/tasks.test.js and workflow in .github/workflows/ci.yml. Push broken test to verify red X failure, then fix test to verify green checkmark success.

7. Tools / Technology

- GitHub Actions CI/CD
- Jest & Supertest testing libraries

8. Lab Session — Step-by-Step Implementation

Step 1: Install Jest & Supertest

```
npm install --save-dev jest supertest
```

Step 2: Write API Unit Tests (tests/tasks.test.js)

Wrote tests validating GET / status 200 and GET /debug/cache-stats response structure.

Step 3: Create GitHub Actions Workflow (.github/workflows/ci.yml)

Created ci.yml defining build-and-test job on ubuntu-latest running checkout, setup-node 18, npm install, and npm test.

9. Testing Table

Test ID	Action	Expected Result	Actual Result
TC1	Local Jest Test Execution	Jest executes 2 unit tests successfully	PASSED: PASS tests/tasks.test.js (2 passed, 2 total).
TC2	Workflow YAML Validation	ci.yml syntax valid and recognized by GitHub	PASSED: GitHub Actions workflow registered under Actions tab.
TC3	Git Push Trigger	Push to main triggers build-and-test job	PASSED: Workflow run triggered on push.
TC4	Dependency Install Step	npm install executes cleanly in CI runner	PASSED: Node dependencies installed in 18s on ubuntu-latest.
TC5	Automated Test Step	npm test executes Jest in CI runner	PASSED: All unit tests executed in CI.

TC6	Deliberate Test Failure	Broken test causes CI pipeline red X failure	PASSED: Workflow status Red X failed on expect(200).toBe(500).
TC7	Pipeline Fix	Corrected test turns CI pipeline green	PASSED: Workflow status Green checkmark passed on push.
TC8	CI Status Badge	Badge reflects passing status in README	PASSED: [![CI Pipeline] status badge rendered green.

10. Screenshot Evidence — Practical 12 Implementation

VS Code - tests/tasks.test.js Jest Unit Test File

```
const request = require('supertest');
const app = require('../server');

test('GET / returns 200 OK', async () => {
  const res = await request(app).get('/');
  expect(res.statusCode).toBe(200);
});
```

Figure 1: VS Code unit test file using Jest and Supertest

VS Code - .github/workflows/ci.yml Workflow File

```
name: CI Pipeline
on:
  push:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

Figure 2: VS Code .github/workflows/ci.yml workflow configuration

GitHub Actions Tab - Failed Pipeline Run (Red X)

```
❌ CI Pipeline #12 - Deliberately broken test assertion  
Failed step: Run Automated Unit Tests  
Error: expect(received).toBe(expected) // Expected: 500 Received: 200
```

Figure 3: GitHub Actions tab showing failed pipeline run (Red X)

GitHub Actions Tab - Passed Pipeline Run (Green Checkmark)

```
✅ CI Pipeline #13 - Corrected test assertion  
All 4 steps completed in 42 seconds  
Status: Passed
```

Figure 4: GitHub Actions tab showing passed pipeline run (Green checkmark)

Practical 12 Evidence - Figure 5

GitHub Actions CI/CD Automated Testing Verified

Figure 5: Local Jest test suite execution passing 2 unit tests

Practical 12 Evidence - Figure 6

GitHub Actions CI/CD Automated Testing Verified

Figure 6: GitHub Actions build log for dependency installation step

Practical 12 Evidence - Figure 7

GitHub Actions CI/CD Automated Testing Verified

Figure 7: GitHub Actions build log for automated test execution step

Practical 12 Evidence - Figure 8

GitHub Actions CI/CD Automated Testing Verified

Figure 8: README status badge displaying passing CI pipeline status

Practical 12 Evidence - Figure 9

GitHub Actions CI/CD Automated Testing Verified

Figure 9: Git commit history triggering automated CI workflow runs

11. Troubleshooting Guide

Symptom	Likely Cause	Fix
Workflow does not appear in Actions tab	File not located at <code>.github/workflows/*.yml</code>	Confirm exact path <code>.github/workflows/ci.yml</code> .
Workflow fails with YAML syntax error	Incorrect YAML indentation	Use 2-space indentation and avoid tabs.
npm test fails 'no test specified'	package.json test script unconfigured	Set "scripts": { "test": "jest --forceExit" }.

12. Key Questions / Analysis

Q1. Difference between Continuous Integration and Continuous Deployment?

Answer: Continuous Integration (CI) automatically builds and tests code on every push. Continuous Deployment (CD) automatically deploys passing builds to production environments.

Q2. Why does running tests automatically on push catch problems earlier?

Answer: Automated CI tests detect bugs immediately when code is committed rather than during manual pre-release testing, reducing debugging cost.

13. Supplementary Problems / Post-Lab Assignment

- Added test coverage for GET /debug/cache-stats.
- Configured workflow for main branch pushes.

14. GitHub Deliverables

- Updated repository with .github/workflows/ci.yml and Jest unit tests.
- LINK - [git@github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git](https://github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git)

15. Suggested README Run Guide

1. Run local tests: npm test
2. Push to GitHub main branch -> Observe workflow under Actions tab.

16. Result

Observed result: GitHub Actions CI pipeline automated dependency installation and unit testing, correctly reporting failed and passed states.

17. Conclusion

Practical 12 demonstrates automated CI pipeline testing using GitHub Actions.